Dicionário de Dados — Motor de Rateio AutoTruck

Documenta **cada campo do sistema** com **nome**, **origem** (de onde o valor vem) e **cálculo / regra de preenchimento**. Organizado pelo fluxo do motor: **Entradas** (planilhas BASE / RATEIO / PRODUTOS) → **Parâmetros** → **Saídas** (eventos rateados, segmentos, boletos, consolidado, assistente de ajuste, auditoria).

Referências de código entre parênteses (`arquivo:linha`). Toda lógica de cálculo vive em `backend/app/autotruck/` (`engine.py` , `rules.py` , `domain.py` , `ingestao_partes.py`); os contratos de API em `backend/app/schemas/autotruck.py` .

> **Ligação com o relatório Hinova.** As contagens de entrada (cotas/boletos por situação) vêm do bloco *"Valor/Cota por Situação de Boleto"* do FECHAMENTO/BASE exportado da Hinova. O motor lê **a coluna BAIXADO** (cotas efetivamente cobradas e quitadas) — **não** a coluna TOTAL (valor nominal de tabela). Por isso TOTAL e BAIXADO divergem na origem (ver discussão de reconciliação).

Convenções

- **Segmento**: `LEVES` | `MOTOS` (rateio variável, bases separadas) | `PESADOS` (casco caminhão = pote/valor fixo segregado). Derivado do tipo do veículo (`rules.py:57` `segmento_por_tipo`).
- **Moeda**: `float` em reais; arredondamento a 2 casas em todo ponto de saída (`round(x, 2)`).
- **Data**: ISO `YYYY-MM-DD` ; normalizada de `dd/mm/YYYY` na ingestão (`ingestao_partes.py:41` `_data`).
- **Mês de referência** (`mes_ano_ref`): string `MM/YYYY` .

---

1. Entradas — BASE (contagens Hinova/SGA)

`BaseCounts` (`ingestao_partes.py:56`). Lido do bloco com cabeçalho `DESCRIÇÃO/ABERTO/BAIXADO/CANCELADO/TOTAL` da aba BASE/FECHAMENTO.

Campo	Origem	Regra de preenchimento
`cotas_baixadas[seg]`	Coluna **BAIXADO** de *"Cota por Situação de Boleto"*	Nº de cotas em boletos quitados, por segmento. **É a base do denominador de rateio.**
`cotas_abertas[seg]`	Coluna **ABERTO** de *"Cota por Situação"*	Cotas em boletos não pagos (informativo; não entra no denominador purista).
`boletos_baixados[seg]`	Coluna **BAIXADO** de *"Quantidade de Boletos"*	Contagem de boletos quitados.
`boletos_abertos[seg]`	Coluna **ABERTO** de *"Quantidade de Boletos"*	Contagem de boletos em aberto.
`valor_recebido[seg]`	*"Resumo de Boletos"* → *VALOR TOTAL PAGO / VALOR RECEBIDO*	Valor financeiro efetivamente recebido.
`valor_inadimplencia[seg]`	*"Resumo de Boletos"* (aberto/diferença)	Valor não recebido.

> A divergência **TOTAL × BAIXADO** observada no relatório é da origem (Hinova): colunas por situação = *valor lançado no boleto*; TOTAL = *valor nominal do produto*. O motor usa apenas BAIXADO.

---

2. Entradas — RATEIO (eventos/sinistros)

`EventoAT` (`domain.py:30`). Layout `Rateio_POC_Completo.xlsx` lido por `ler_rateio_partes` (`ingestao_partes.py:271`); colunas mapeadas em `_RC` (`ingestao_partes.py:264`).

Campo	Origem	Regra de preenchimento / cálculo
`id_evento`	Gerado	Sequencial `UP-00001…` na ordem de leitura (`ingestao_partes.py:319`).

Campo	Origem	Regra de preenchimento / cálculo
`protocolo`	Coluna 1 (Protocolo)	Texto do protocolo do sinistro.
`placa`	Coluna 6 (Placa)	—
`segmento`	Marcador de seção na coluna A (`"MOTOS ASSOCIADOS"` , `"PESADOS TERCEIROS"` …)	Detectado pelo cabeçalho de seção (`ingestao_partes.py:287`).
`envolvimento`	Marcador de seção	`ASSOCIADO` (custo) se a seção termina em "ASSOCIADOS"; `TERCEIRO` (recuperação) se "TERCEIROS" (`ingestao_partes.py:294`).
`tipo_evento`	Coluna B "INDENIZAÇÃO INTEGRAL/PARCIAL"	`INTEGRAL` ou `PARCIAL` lido da linha (`ingestao_partes.py:301`). Pode ser reclassificado pela regra dos 75% no motor.
`valor_evento`	Coluna 11 (Valor Rateado)	Valor bruto do evento; base de custo do PARCIAL.
`valor_fipe`	Coluna 11 (derivado)	INTEGRAL: = valor rateado. PARCIAL: `valor_rateado / 0,5` (FIPE alto para **não** disparar a regra dos 75% de forma espúria) (`ingestao_partes.py:315-317`).
`orcamento_reparo`	—	`0.0` no layout atual (base da regra dos 75% quando informado).
`data_aviso`	Coluna 8 (Data Comunicação)	Chave de ordenação **FIFO**; normalizada para ISO.
`franquia`	—	`0.0` (deduz do custo a ratear quando informado).
`valor_salvado`	—	`0.0` (crédito de salvado).
`salvado_liquidado`	—	`False` . Crédito do salvado **só** abate no mês da liquidação financeira (`domain.py:43`).
`postergado`	Estado entre meses	`True` se o evento foi cortado em mês anterior → **prioridade FIFO #1 absoluta** (`domain.py:44` , `engine.py:134`).

3. Entradas — Veículo

`VeiculoAT` (`domain.py:17`). Na PoC Remonta são **sintetizados** a partir das contagens da BASE (`sintetizar_veiculos` , `ingestao_partes.py:374` , FIPE=5000 ⇒ 1 cota cada).

Campo	Origem	Regra de preenchimento / cálculo
`id_veiculo`	Cadastro / gerado	Identificador único.
`placa`	Cadastro	—
`segmento`	Tipo do veículo	`segmento_por_tipo(tipo_veiculo)` (`rules.py:57`).
`tipo_veiculo`	Cadastro	AUTOMOVEL, UTILITÁRIO, PREMIUM, MOTOCICLETA, CAMINHAO, SEMI REBOQUE.
`valor_fipe`	Cadastro/FIPE	Base do nº de cotas e do valor integral em perda total.
`status_ativo`	SGA (foto de fechamento)	Ativo no mês.
`adimplente`	Situação de boleto (BAIXADO)	Pagou → **entra no denominador purista** (`engine.py:82`).
`servicos_adicionais`	Contratos (Assist 24h, Vidros…)	Soma fixa somada direto no boleto.
`cobertura_terceiros`	Produto Danos a Terceiros	`NENHUM` | `50K` | `100K` — define o preço do pote terceiros.

4. Parâmetros do motor

`ParametrosAT` (`rules.py:12`). Mudam limiares **globalmente**, nunca a forma do cálculo.

Parâmetro	Default	Significado / uso
`cota_base`	`1,0`	Cota de veículo ≤ R$10.000.
`passo_cota`	`0,5`	Incremento de cota a cada faixa de R$10k.
`faixa_fipe`	`10.000`	Tamanho da faixa de progressão.

Parâmetro	Default	Significado / uso
`fipe_cota_especial`	`450.000`	Acima disso → cota especial (não progride linear).
`multiplicador_especial`	`20,0`	Cota fixa do high-end (faixa permitida 15–20).
`threshold_perda_total`	`0,75`	Regra dos 75%: orçamento > 75%·FIPE ⇒ INTEGRAL.
`cota_alvo_min` / `cota_alvo_max`	`30` / `35`	Faixa de estabilização da cota.
`var_trimestral_max`	`0,05`	Variação > ±5% vs média 3M dispara alerta/ajuste.
`pct_volatilidade`	`0,25`	1 sinistro > 25% do pool ⇒ alerta de instabilidade.
`arrecadacao_terceiros_estimada`	`500.000`	Fallback do pote Terceiros.
`preco_terceiros_50k` / `preco_terceiros_100k`	`20` / `30`	Preço mensal do produto Danos a Terceiros por faixa.
`preco_casco_caminhao`	`0,0`	>0 ativa arrecadação real do pote caminhão (valor fixo/assoc).

Cálculos derivados de parâmetro: - **Cotas por FIPE** (`cotas_por_fipe` , `rules.py:66`): se `fipe > 450k` → `(20, especial=True)` ; senão `cota_base + passo_cota·max(0, ⌈fipe/10k⌉−1)` . - **Regra dos 75%** (`aplica_regra_75` , `rules.py:78`): `fipe>0 and orcamento > 0,75·fipe` . - **Preço terceiros** (`preco_cobertura_terceiros` , `rules.py:43`): `NENHUM→0` , `50K→20` , `100K→30` .

5. Saída — Evento Rateado

`EventoRateadoAT` (`domain.py:71`) / `EventoRateadoATOut` . Um por evento processado; ordenado por FIFO (`engine.py:131`).

Campo	Origem	Cálculo
`mes_ano_ref`	Parâmetro de execução	—
`id_evento` , `protocolo` , `segmento` , `envolvimento` , `data_aviso`	Evento de entrada	Cópia.
`tipo_evento`	Evento + regra 75%	Tipo **efetivo** após reclassificação (`engine.py:95`).
`reclassificado_75pct`	Regra dos 75%	`True` se PARCIAL virou INTEGRAL por `aplica_regra_75` (`engine.py:96`).
`valor_evento`	Evento	Valor bruto (2 casas).
`custo_a_ratear`	Calculado	INTEGRAL→ `valor_fipe` ; PARCIAL→ `valor_evento` (`_custo_evento` , `engine.py:41`).
`ordem_fifo`	Calculado	Posição na fila: postergado primeiro → `data_aviso` ASC → INTEGRAL antes de PARCIAL → maior custo → id (`engine.py:131-140`).
`status`	Calculado	`ALOCADO` ; vira `POSTERGADO` em PESADOS/ASSOCIADO quando estoura o teto do pote caminhão (`engine.py:148-152`).

6. Saída — Resultado por Segmento

`SegmentoResultado` (`domain.py:48`) / `SegmentoOut` . Loop de apuração `engine.py:172` (variáveis) e `engine.py:226` (PESADOS).

Campo	Origem	Cálculo
`segmento`	—	LEVES / MOTOS / PESADOS.
`modo`	—	`RATEIO_VARIAVEL` (LEVES/MOTOS) ou `POTE_FIXO` (PESADOS).
`custo_integrais` (ci)	Soma	Σ custo dos eventos ASSOCIADO INTEGRAL do segmento (`_acc` , `engine.py:111`).
`custo_parciais` (cp)	Soma	Σ custo dos eventos ASSOCIADO PARCIAL.
`deducao_salvados` (ds)	Soma	Σ `valor_salvado` **apenas** dos salvados liquidados (`engine.py:115`).
`deducao_franquias` (df)	Soma	Σ `franquia` dos eventos do segmento.

Campo	Origem	Cálculo
`recuperacao_terceiros` (rt)	Soma	Σ custo dos eventos com envolvimento TERCEIRO — **NÃO abate a cota** (vai pro pote Terceiros) (`engine.py:108`).
`custo_a_ratear`	Calculado	`ci + cp − ds − df` (`engine.py:177`).
`cotas_ativas`	BASE	Σ cotas de veículos ativos **e** adimplentes do segmento (`engine.py:83`).
`valor_cota`	Calculado	**`custo_a_ratear / cotas_ativas`** (0 se sem cotas) (`engine.py:181`). É a "cota" do mês.
`arrecadado`	Pote	`0,0` nos variáveis; arrecadação do pote nos PESADOS.
`reserva_ou_deficit`	Calculado	`0` nos variáveis; `arrecadado − custo` nos PESADOS (`engine.py:225`).
`alerta_volatilidade`	Calculado	`True` se maior sinistro > 25% do bruto do pool (`engine.py:183`).
`faixa_estabilizacao`	Calculado	`ABAIXO` (<30), `OK` (30–35), `ACIMA` (>35) (`engine.py:189-194`).
`sugestao`	Calculado	Texto-guia conforme faixa + alertas (volatilidade / variação trimestral) (`engine.py:190-201`).
`diferenca_alvo_vs_alocado`	Invariante	`custo_a_ratear − custo_alocado` (deve ser 0; **invariante de auditoria**) (`engine.py:179`).
`media_cota_3m`	Histórico	Média das cotas dos últimos 3 meses (`engine.py:186`).
`variacao_vs_media`	Calculado	`(valor_cota − media_3m) / media_3m` (`engine.py:187`).
`alerta_variacao_trimestral`	Calculado	`True` se `|variacao| > 5%` e há histórico (`engine.py:188`).

7. Saída — Boleto

`BoletoAT` (`domain.py:87`) / `BoletoATOut` . Um por veículo (`engine.py:265`).

Campo	Origem	Cálculo
`id_veiculo` , `placa` , `mes_ano_ref` , `segmento` , `valor_fipe`	Veículo	Cópia.
`qtd_cotas`	Calculado	`cotas_por_fipe(valor_fipe)` (`engine.py:266`).
`cota_especial`	Calculado	`True` se FIPE > teto especial.
`custo_rateio`	Calculado	`qtd_cotas · valor_cota` do segmento (só se elegível e segmento variável) (`engine.py:268`).
`custo_pote_caminhao`	Calculado	PESADOS elegível: `preco_casco_caminhao` (N4) ou rateio per-capita do pote (`engine.py:269-274`).
`custo_pote_terceiros`	Calculado	Preço da cobertura contratada, ou per-capita do pote estimado (`engine.py:275-280`).
`servicos_adicionais`	Veículo	Soma fixa de serviços contratados.
`valor_total_boleto`	Calculado	`custo_rateio + custo_pote_caminhao + custo_pote_terceiros + servicos_adicionais` (`engine.py:281`).
`gera_boleto`	Calculado	`status_ativo AND adimplente` (`engine.py:267`).

8. Saída — Consolidado do mês

`ConsolidadoAT` (`domain.py:138`) / `ConsolidadoATOut` . Montado em `engine.py:296` .

Campo	Cálculo
`cota_leves` / `cota_motos`	`valor_cota` de cada segmento variável.
`custo_a_ratear_leves` / `_motos`	`custo_a_ratear` por segmento.
`bruto_leves` / `_motos`	`ci + cp` (antes de salvados/franquias) (`engine.py:182`).
`recuperacao_terceiros_leves` / `_motos`	`rt` por segmento.
`pote_caminhao_arrecadado`	`preco_casco_caminhao·n_pes` ou `valor_fixo·n_pes` ou estimada (`engine.py:124-129`).

Campo	Cálculo
`pote_caminhao_custo`	`ci+cp-ds-df` dos PESADOS (`engine.py:224`).
`pote_caminhao_reserva`	`arrecadado − custo` (`engine.py:225`).
`pote_terceiros_arrecadado`	Σ preço das coberturas elegíveis, ou `valor_fixo·n` , ou estimada (`engine.py:250-257`).
`pote_terceiros_custo`	Σ recuperações de terceiros (todos os segmentos + PESADOS) (`engine.py:245`).
`pote_terceiros_reserva`	`arrecadado − custo` (`engine.py:258`).
`total_veiculos`	Contagem total de veículos.
`total_veiculos_elegiveis`	Ativos **e** adimplentes (`engine.py:85`).
`total_cotas_leves` / `_motos`	Σ cotas ativas por segmento.
`eventos_alocados`	Nº eventos com `status=ALOCADO` (`engine.py:316`).
`eventos_postergados`	Nº cortados pelo teto do pote caminhão (`engine.py:317`).
`eventos_reclassificados_75pct`	Nº PARCIAL→INTEGRAL pela regra dos 75%.
`diferenca_total`	Σ `diferenca_alvo_vs_alocado` dos variáveis (`engine.py:293`).
`invariante_ok`	`|diferenca_total| < 0,01` (`engine.py:320`).
`alerta_variacao_trimestral`	`True` se algum segmento estourou ±5% vs média 3M.

Saídas agregadas na API (`ResultadoATOut` , `schemas:189`): `total_eventos` , `total_boletos` , `total_boletos_a_emitir` (só `gera_boleto`), `receita_total_esperada` (Σ `valor_total_boleto` dos boletos a emitir) (`api/autotruck.py:58-66`).

9. Saída — Assistente de Ajuste

`SugestaoAjuste` (`domain.py:122`) / `SugestaoAjusteOut` . Calculado por `sugerir_ajuste` (`engine.py:337`). **Alvo independente por segmento** (`alvos_cota`).

Campo	Cálculo
`segmento`	LEVES / MOTOS.
`cota_atual`	`valor_cota` apurado do segmento.
`alvo_cota`	`alvos_cota[seg]` → `alvo_cota` global → default (`media_3m·1,05` ou R$35) (`engine.py:359` , `_alvo_default` `engine.py:329`).
`precisa_ajuste`	`True` se `cota_atual > alvo` .
`reducao_necessaria`	`(cota_atual − alvo) · cotas_ativas` (`engine.py:371`).
`eventos_sugeridos`	Conjunto **mínimo** a postergar: candidatos ASSOCIADO ALOCADO INTEGRAL (PARCIAL só se `incluir_parciais`), ordenados por valor DESC, desempate mais recente (`engine.py:372-394`).
`reducao_obtida`	Σ custo dos eventos escolhidos.
`cota_resultante`	`(custo_a_ratear − reducao_obtida) / cotas_ativas` (`engine.py:396`).
`atingiu_alvo`	`cota_resultante ≤ alvo` (`engine.py:397`).
`mensagem`	Texto-guia do resultado.

10. Auditoria e Fechamento

Campo	Estrutura	Origem / regra
`AuditLog.acao`	`domain.py:104`	`EXCLUSAO_MANUAL` / `INCLUSAO_MANUAL` / postergação no ajuste.
`AuditLog.justificativa`	—	**Obrigatória** e não-vazia; senão a exclusão é ignorada (`engine.py:67`).
`AuditLog.timestamp`	—	ISO UTC do processamento.
`FotoFechamento.hash_insumos`	`schemas:167`	Hash dos insumos da foto (integridade).

Campo	Estrutura	Origem / regra
`IntegridadeOut.intacto`	`schemas:183`	`hash_foto == hash_atual` (detecta adulteração pós-homologação).

Invariante central

Para cada segmento variável: **`custo_a_ratear − custo_dos_eventos_alocados = 0`** (análogo ao "alvo vs alocado = 0" do ELLO).
Consolidado: `invariante_ok` exige `|diferenca_total| < 0,01` (`engine.py:10-11` , `engine.py:320`).